

AI Advent Challenge #8

Неделя 1. День 1: базовые основы ИИ, LLM, промптинг и параметры моделей

Исправленный и дополненный конспект по лекции + практическая памятка для выполнения задания

Главная мысль документа

Цель первой недели - научиться управлять LLM: писать промпты, менять параметры генерации, сравнивать модели, понимать экономику токенов и собирать первый работающий LLM-playground для демонстрации результата.

Параметр	Значение
Курс	AI Advent Challenge #8
Неделя	1
День	1
Тема	AI, LLM, prompting, agenting, параметры моделей
Что добавлено	Исправления терминов, практическая матрица экспериментов, чек-листы, шаблоны промптов, рекомендации для видео и README

0. Что было исправлено и дополнено

Блок	Что исправлено / добавлено	Почему важно
Top-P	Уточнено, что top_p обычно означает nucleus sampling: модель берёт минимальный набор наиболее вероятных токенов, суммарная вероятность которых достигает p.	В исходной формулировке top_p выглядел как простой порог по вероятности каждого токена. Это полезная интуиция, но технически неточно.
Top-K	Уточнено, что top_k ограничивает выборку k наиболее вероятными токенами до случайного выбора следующего токена.	Помогает правильно объяснить разницу top_p и top_k в видео/README.
Max tokens	Разделены max_tokens как лимит вывода и context length как лимит вход + история + будущий вывод.	Это частая ошибка: max_tokens не равно размеру всего контекстного окна.
Temperature	Уточнено, что в API низкая температура обычно делает ответ более стабильным, высокая - более разнообразным; диапазон зависит от модели/провайдера.	Нельзя обещать идеальную детерминированность даже при temperature=0.
Seed	Добавлено, что seed работает только если провайдер/модель поддерживает его, и не гарантирует вечную воспроизводимость после обновления модели.	Важно для экспериментов: seed - не абсолютная гарантия одинаковых ответов.
Frequency / presence penalty	Разделена логика: frequency penalty штрафует частые повторы, presence penalty штрафует сам факт появления токена/темы.	Так проще осмысленно тестировать параметры.
Chain of Thought	Добавлено практическое правило: для рабочих задач просить не скрытую цепочку мыслей, а краткое обоснование, план, критерии проверки и финальный ответ.	Это безопаснее и лучше подходит для сдачи задания.
Практика недели	Добавлены структура LLM playground, матрица экспериментов, сценарий видео, README-чеклист.	Конспект превращается в рабочий документ, по которому можно сразу делать задание.

1. Короткое резюме лекции

- 🔗 **Неделя 1 - исследовательская.** Нужно не строить сложную систему, а разобраться, как LLM реагирует на промпты и параметры.
- 🔗 **Главный навык - управление AI.** В челлендже важно не вручную писать весь код, а учиться ставить задачи harness/code-assistant инструментам.
- 🔗 **Видео важнее красоты кода.** Для сдачи нужен работающий результат, который можно показать на экране.
- 🔗 **LLM - не магия.** Ответ формируется через вероятностный выбор токенов, контекст, системные инструкции и параметры генерации.
- 🔗 **Агент - это не просто чат.** Агент имеет цикл: perception -> memory -> reasoning -> planning -> action -> feedback.
- 🔗 **Параметры надо потрогать руками.** Temperature, top_p, top_k, max_tokens, seed и penalties лучше понять через сравнение ответов на одинаковых промптах.

Практический вывод

Идеальный артефакт для первого задания - небольшой LLM Playground: CLI или мини-приложение, которое отправляет одинаковые промпты в модель с разными параметрами и сохраняет результаты экспериментов.

2. Карта документа

1. Контекст первой недели и методический фокус курса.
2. Как разработчики используют AI и почему ему не доверяют.
3. Где AI применять в жизни и работе разработчика.
4. Словарь терминов: code-assistant, агент, LLM, токен, контекст, субагент, skill, swarm.
5. Техники prompting: простой prompt, reasoning, metaprompting, swarm prompting.
6. Agenting: цикл perception -> memory -> reasoning -> planning -> action -> feedback.
7. Параметры LLM: temperature, top_p, top_k, max_tokens, seed, penalties.
8. Роли сообщений: system, user, assistant, tool/developer в современных API.
9. Чем отличаются модели: параметры, скорость, контекст, reasoning quality, экономика.
10. Практическое задание недели: как собрать LLM Playground и что показать в видео.

3. Контекст первой недели

Первая неделя посвящена базовым основам работы с искусственным интеллектом и промптингом. Это подготовительная неделя: задача не в том, чтобы сразу сделать сложную агентную систему, а в том, чтобы научиться наблюдать поведение моделей и управлять им.

- 🔗 понять, как разработчики используют AI в повседневной работе;
- 🔗 разобраться, почему AI уже используется массово, но доверие к нему остаётся ограниченным;
- 🔗 освоить базовый словарь: LLM, token, context window, code-assistant, agent, subagent, skill, swarm;
- 🔗 потрогать техники prompting и agenting на практике;
- 🔗 проверить, как параметры генерации меняют ответ модели;
- 🔗 собрать первые эксперименты, которые можно показать в видео.

Фокус челленджа

В этом курсе важно не просто написать код, а научиться работать как оператор AI-разработки: ставить задачу, проверять результат, управлять контекстом, выбирать модель, контролировать стоимость и быстро доводить прототип до демонстрации.

4. Использование AI в IT: главный конфликт

Лекция начинается с исследований использования AI в IT. Общий вывод: разработчики уже используют AI, но часто не доверяют ему полностью. Поэтому они продолжают читать сгенерированный код, перепроверять ответы, запускать тесты, делать pull request review и CI/CD-проверки.

Наблюдение	Смысл
Почти все разработчики хотя бы раз использовали AI	Но это не означает ежедневную агентную разработку. Часто это просто ChatGPT для функции, сниппета или объяснения.
Доля кода, затронутого AI, растёт	AI постепенно перестаёт быть игрушкой и становится частью production workflow.
Зрелые компании быстро внедряют AI-автоматизацию	Конкурентное преимущество получают те, кто умеет встроить AI в процессы, а не просто задавать вопросы в чат.
Senior-разработчики доверяют AI меньше	Эксперт видит ошибки модели быстрее и часто

	разочаровывается, если ожидает от неё уровень своего многолетнего опыта.
--	--

Баланс доверия

Правильная стратегия не в том, чтобы слепо верить модели, и не в том, чтобы всё писать руками. Правильная стратегия - давать AI большую часть работы, но проверять результат через критерии: тесты, запуск, логика, безопасность, стоимость, UX и демонстрацию.

5. Где использовать AI в жизни разработчика

5.1. Code generation для pet-проектов

AI снижает порог входа для личных проектов. Идеи, которые раньше требовали команды, дизайнера, backend/frontend-разработчиков, тестировщика и бюджета, теперь можно прототипировать через code-assistant инструменты.

- 🔗 аналог YouTube/VK/личного SaaS в минимальной версии;
- 🔗 Telegram-бот;
- 🔗 мини-CRM;
- 🔗 сервис для заметок, поиска, анализа документов;
- 🔗 личный AI-ассистент под конкретную задачу.

5.2. Поиск, образование и верификация

AI полезен не только как генератор текста, но и как продвинутый интерфейс к знаниям. Однако важный принцип: модель помогает найти и структурировать материал, а финальный вывод должен делать человек.

- 🔗 просить список источников, терминов и направлений поиска;
- 🔗 сравнивать несколько объяснений;
- 🔗 просить объяснение на разных уровнях сложности;
- 🔗 проверять противоречия между источниками;
- 🔗 отделять факты от интерпретаций.

5.3. Собственные агенты

Агенты - один из центральных мотивов челленджа. В дальнейшем будут встречаться MCP, RAG, мультиагентные системы, Agent-to-Agent подходы и разные способы встраивания LLM в приложение.

5.4. Автоматизация жизни

AI хорошо подходит для сценариев, где нужно учитывать контекст, паттерны и мягкие правила. Например, умный дом может реагировать не только на время суток, но и на день недели, сезон, присутствие людей дома, привычки и накопленную историю поведения.

6. Где использовать AI на работе

Направление	Что можно автоматизировать	Практическая ценность
Code generation agents	Генерация типового кода, миграции, шаблоны, CRUD, SDK, интеграции.	Ускорение разработки и снижение рутины.
Task management	Разбор задач, уточнение acceptance criteria, декомпозиция, статус-апдейты.	Быстрая ачивка для команды, особенно если процессы хаотичные.

Analytics automation	Сбор требований, анализ логов, отчётов, метрик, формирование summary.	Экономия времени аналитиков и менеджеров.
Code review/testing	Поиск смысловых ошибок, однотипных багов, генерация тест-кейсов.	Дополнительный слой качества поверх линтеров и CI.
Deployment/DevOps	Генерация конфигов, проверка пайплайнов, объяснение ошибок деплоя.	Быстрее находить проблемы в инфраструктуре.
Fine-tuning/optimization	Дообучение, адаптация, ускорение inference, снижение затрат.	Более advanced-направление, требует сильной математики и ML-инженерии.

7. Эволюция использования AI

Период / уровень	Типичное использование	Ограничение
Базовый этап	ChatGPT как замена Stack Overflow: попросить функцию, объяснение, regex, маленький сниппет.	AI не встроен в рабочий процесс, всё остальное пишется руками.
Промежуточный этап	Задачи для модели: экран по дизайну, boilerplate, перевод кода, data-классы, первые MCP.	AI всё ещё получает только простые задачи, ревью остаётся полностью ручным.
Agentic development	Большая часть кода пишется агентами; несколько проектов идут параллельно; используются subagents, skills, swarm.	Нужно уметь управлять контекстом, стоимостью, проверкой и качеством результата.

8. Словарь терминов

Термин	Короткое определение	Пример
Code-assistant / harness	Инструмент, который помогает писать и изменять код.	Cursor, Claude Code, Codex, OpenCode, Copilot.
Agent	AI-сущность внутри приложения или рабочего процесса, которая выполняет задачу в заданном контексте.	Агент, который анализирует питание пользователя или отвечает по статусу заказа.
LLM	Large Language Model, большая языковая модель.	Модель, к которой подключается code-assistant или backend через API.
Token	Единица текста, на которую модель разбивает вход и выход.	Слово, часть слова, знак препинания - зависит от токенизатора.
Context window	Максимальный объём токенов, который модель может учитывать в одном запросе/сессии.	История диалога + файлы + инструкции + будущий вывод.
Subagent	Специализированный агент под отдельную область ответственности.	Backend-agent, frontend-agent, test-agent.
Skill	Маленькое действие или навык, который агент может использовать.	Запустить CLI-команду, прочитать файл, вызвать API, проверить тесты.
Swarm / рой агентов	Много мелких агентов, каждый из	Один агент проектирует API, второй

	которых решает часть задачи, а главный агент собирает результат.	пишет тесты, третий проверяет безопасность.
Prompting	Управление моделью через текстовые инструкции, контекст и примеры.	Задать роль, входные данные, ограничения, формат ответа.

9. Техники prompting

9.1. Простой prompt

Самый простой вариант: пользователь пишет один запрос и получает ответ. В современных AI-системах даже такой запрос часто проходит через дополнительные внутренние инструкции, маршрутизацию, фильтры, суммаризацию и другие этапы.

9.2. Reasoning-oriented prompt

В лекции упоминается Chain of Thought как подход, когда модель просят не просто дать ответ, а объяснить ход рассуждения. Практически лучше просить не длинную скрытую цепочку мыслей, а контролируемый формат: план, краткое обоснование, проверки, финальный ответ.

Плохой вариант:

"Думай пошагово и покажи весь ход мыслей".

Лучший рабочий вариант:

"Сначала дай краткий план решения, затем финальный ответ. Отдельно перечисли проверки, по которым можно убедиться, что ответ корректен".

9.3. Deliberate reasoning / multi-perspective prompting

Один и тот же вопрос рассматривается с разных сторон. Это полезно, когда есть риск одностороннего решения.

Рассмотри задачу с трёх позиций:

1. backend-разработчик;
2. архитектор;
3. QA-инженер.

Для каждой позиции дай 3 риска и 3 рекомендации. Потом сведи всё в единый план действий.

9.4. Metaprompting

Метапромптинг - это когда одна модель помогает подготовить качественный prompt для другой модели или для следующего шага работы.

Ты - prompt engineer. Помоги мне составить промпт для code-assistant.

Цель: сделать Python CLI для экспериментов с LLM-параметрами.

Сначала задай до 7 уточняющих вопросов, затем выдай финальный промпт для Cursor/Claude Code.

9.5. Swarm prompting

Несколько моделей или несколько независимых запросов решают одну задачу, затем пользователь или отдельный агент сравнивает ответы и собирает итог. Это можно делать даже без полноценного агентного фреймворка.

- ⌘ один запрос - как backend engineer;
- ⌘ второй запрос - как product engineer;
- ⌘ третий запрос - как security reviewer;
- ⌘ четвёртый шаг - свести ответы, убрать противоречия и получить план.

10. Agenting: чем агент отличается от обычного чата

Агент - это не просто LLM-ответ на один вопрос. Агент проходит цикл обработки задачи:

perception -> memory -> reasoning -> planning -> action -> feedback

Стадия	Что означает	Пример в приложении
Perception	Входные данные: prompt, API-ответ, файл, данные устройства, событие.	Пользователь отправил текст задачи или загрузил CSV.
Memory	Долговременные данные: база, файлы, история, профиль, правила проекта.	Агент читает README, прошлые решения, настройки проекта.
Reasoning	Обработка perception + memory и получение вывода.	Агент понимает, какие файлы надо изменить.
Planning	Разбиение цели на шаги и стратегия выполнения.	Сначала создать клиент API, затем CLI, затем тесты.
Action	Видимое действие: текст, код, вызов API, команда, изменение файла.	Агент пишет main.py и запускает pytest.
Feedback	Сигнал качества: ошибка, тесты, комментарий пользователя, метрика.	Пользователь пишет: "не то", тест падает, агент исправляет.

Практический смысл

Когда ты строишь даже простой LLM playground, уже можно показать элементы agenting: input пользователя, конфигурация параметров, вызов модели, сохранение результатов, сравнение ответов и вывод.

11. Параметры LLM: исправленная версия

Параметры генерации управляют тем, как модель выбирает следующий токен. Они не делают модель "умнее" напрямую, но меняют стиль, стабильность, разнообразие и стоимость ответа.

Параметр	Что делает	Когда уменьшать	Когда увеличивать / использовать
temperature	Регулирует случайность выбора токена. Низкая температура - стабильнее, высокая - разнообразнее.	Точные ответы, код, инструкции, JSON, расчёты.	Идеи, нейминг, тексты, brainstorming.
top_p	Nucleus sampling: оставляет минимальный набор самых вероятных токенов с суммарной вероятностью p.	Когда нужен более предсказуемый ответ.	Когда нужен более широкий выбор вариантов.
top_k	Ограничивает выбор k наиболее вероятными токенами.	Когда нужно жёстче ограничить выбор модели.	Когда модель должна иметь больше вариантов для генерации.
max_tokens	Ограничивает длину вывода. Это не то же самое, что полный context length.	Когда нужен короткий ответ и контроль стоимости.	Когда нужен длинный отчёт, конспект, код, таблицы.
seed	Задаёт зерно случайности, если поддерживается провайдером.	Не "уменьшают", а фиксируют для эксперимента.	Используют для более воспроизводимых сравнений.

frequency_penalty	Штрафует частые повторы пропорционально количеству повторений.	Когда повторы допустимы или важны.	Когда модель закликивается на одинаковых словах.
presence_penalty	Штрафует сам факт появления токена/темы, стимулируя новые темы.	Когда нужно держаться одной темы.	Когда нужен более широкий набор идей.

Важное исправление по max_tokens

context length - это общий лимит: system instructions + user prompt + история + документы + tool results + будущий output. max_tokens чаще всего ограничивает именно output. Поэтому "поставить max_tokens побольше" не равно "дать модели больше памяти".

Важное исправление по top_p

top_p обычно не означает "отсекаем все токены ниже вероятности 0.5". Это скорее "берём верхушку распределения вероятностей, пока их суммарная вероятность не достигнет p".

12. Роли сообщений: system, user, assistant и другие

В лекции выделяются три базовые роли: system prompt, user prompt, assistant prompt. Это правильная основа, но в современных инструментах могут встречаться и дополнительные роли: developer/tool/function. Для челленджа достаточно понимать базовую логику.

Роль	Смысл	Пример
system	Главная рамка поведения модели: роль, стиль, запреты, формат.	"Ты опытный Python backend engineer. Отвечай структурно."
user	Сообщение пользователя: задача, данные, вопрос, ограничение.	"Сделай CLI для сравнения temperature."
assistant	Ответ модели, который сохраняется в истории.	Модель предлагает план, код, объяснение.
tool/function	Результат вызова инструмента, если агент умеет пользоваться tools.	Результат чтения файла, выполнения тестов, HTTP-запроса.
developer	Дополнительный слой инструкций в некоторых системах/API.	Правила проекта, стиль кода, ограничения команды.

```
[
  {"role": "system", "content": "Ты эксперт по Python и LLM-интеграциям."},
  {"role": "user", "content": "Сделай план CLI для экспериментов с temperature, top_p и max_tokens."}
]
```

Что важно для Cursor/Claude Code

В code-assistant средах аналогом system/developer prompt часто становятся project rules, cursor rules, CLAUDE.md, AGENTS.md, README, architecture docs и другие файлы, которые агент читает как правила проекта.

13. Чем отличаются модели

Критерий	Что означает	Как выбирать
Количество параметров	Размер и сложность модели, но не абсолютная гарантия качества.	Не выбирать только по размеру. Смотреть на качество под задачу.
Inference speed	Скорость генерации токенов.	Для интерактивных CLI/чатов важна

		высокая скорость.
Context length	Сколько токенов модель может учитывать.	Для больших кодовых баз и документов нужен длинный контекст, но он дороже и не всегда качественно используется.
Reasoning quality	Качество анализа, планирования, вывода, исправления ошибок.	Для архитектуры и сложного кода брать сильные reasoning/code модели.
Экономика	Цена входных и выходных токенов.	Для production API лучше дешёвые модели, если качество достаточно. Для разработки можно брать сильнее и дороже.

Главный критерий выбора модели

Модель выбирают не “самую большую” и не “самую модную”, а подходящую под задачу: коддинг, reasoning, длинный контекст, быстрый ответ, дешёвый API, JSON-вывод, мультимодальность и т.д.

14. Практическое задание недели: что реально сделать

По формулировке задания недели нужно “постучаться” в LLM, получить ответ, поменять параметры, попробовать разные промпты и разобраться, какие возможности дают разные модели. Поэтому оптимальный проект - небольшой LLM Playground.

Минимум для сдачи	Хороший уровень	Отличный уровень
Один Python-скрипт вызывает LLM через API и печатает ответ.	CLI принимает prompt, model, temperature, top_p, max_tokens и сохраняет результаты.	Есть матрица экспериментов, README, результаты в Markdown/JSON, сравнение моделей и выводы.
Один промпт и один ответ.	Несколько промптов: coding, creative, reasoning.	Одинаковые промпты прогнаны на разных параметрах и/или моделях.
Видео показывает запуск.	Видео показывает разницу ответов.	Видео показывает структуру проекта, параметры, результаты, выводы и следующий шаг.

14.1. Рекомендуемая структура репозитория

```

ai-advent-week1-llm-playground/
├── README.md
├── .env.example
├── requirements.txt
├── main.py
├── prompts/
│   ├── coding_task.txt
│   ├── creative_task.txt
│   └── reasoning_task.txt
├── results/
│   ├── experiments.md
│   └── raw_results.json
├── docs/
└── conclusions.md

```

14.2. Матрица экспериментов

Эксперимент	Prompt	temperature	top_p	max_tokens	Что сравнить
E1	coding_task	0.0	1.0	500	Насколько ответ сухой, стабильный, пригодный для кода.
E2	coding_task	0.7	1.0	500	Появляются ли альтернативные идеи или лишняя болтовня.
E3	creative_task	0.2	1.0	700	Не слишком ли скучный ответ для творческой задачи.
E4	creative_task	1.0	1.0	700	Становится ли ответ более креативным и менее предсказуемым.
E5	reasoning_task	0.3	0.8	800	Держится ли модель в рамках и не уходит ли в сторону.
E6	reasoning_task	0.8	0.95	800	Появляются ли новые варианты решения и больше риска ошибок.

14.3. Что записать в results/experiments.md

- какая модель использовалась;
- какой prompt отправлялся;
- какие параметры были выставлены;
- какой ответ получен;
- что изменилось при другой temperature/top_p/max_tokens;
- какой режим лучше для кода, какой для креатива, какой для reasoning;
- сколько примерно токенов ушло, если API возвращает usage.

15. README и видео: что показать на сдаче

Артефакт	Что должно быть
README.md	Цель проекта, как установить, как прописать API key, как запустить, примеры команд, таблица экспериментов, выводы.
.env.example	OPENAI_API_KEY=..., BASE_URL=..., MODEL=... без реальных ключей.
main.py	Читаемый endpoint: аргументы CLI, вызов API, сохранение результата.
results/experiments.md	Сравнение ответов и краткие выводы.

Видео	1) структура проекта; 2) запуск; 3) параметры; 4) разные ответы; 5) выводы.
-------	---

15.1. Сценарий короткого видео

11. Показать README и сказать: “Это LLM playground для экспериментов с параметрами модели”.
12. Показать .env.example без реальных ключей.
13. Запустить первый эксперимент с низкой temperature.
14. Запустить тот же prompt с высокой temperature.
15. Открыть results/experiments.md и показать разницу.
16. Сформулировать вывод: какие параметры лучше для точных задач, какие для творческих.
17. Показать, что код и результаты лежат в репозитории/архиве.

16. Частые ошибки первой недели

Ошибка	Почему плохо	Как сделать правильно
Сделать только чат без сравнения параметров.	Не видно, что ты понял тему лекции.	Обязательно показать минимум 3-6 экспериментов.
Писать много ручного кода и забыть про AI workflow.	Курс учит управлять AI, а не просто программировать.	Генерировать код через Cursor/Claude/Codex и фиксировать выводы.
Не сохранять результаты.	В видео сложно доказать, что было сравнение.	Писать raw_results.json и experiments.md.
Путать max_tokens и context window.	Неправильное объяснение параметров.	В README отдельно объяснить: max_tokens - лимит вывода, context - общий лимит.
Залить API key в GitHub.	Критическая утечка секрета.	Использовать .env, .gitignore, .env.example.
Не записать видео.	Видео - ключевой артефакт сдачи.	Сначала записать короткое демо, потом полировать код.

17. Готовые prompts для экспериментов

17.1. Coding task

Ты опытный Python backend developer.
 Напиши минимальный CLI-план для проекта, который сравнивает ответы LLM при разных значениях temperature, top_p и max_tokens.
 Ответ дай в формате:

1. структура файлов;
2. основные функции;
3. какие ошибки обработать;
4. какие результаты сохранить.

17.2. Creative task

Придумай 10 названий для developer-tool проекта, который помогает сравнивать LLM-параметры.
 Для каждого названия дай короткое объяснение и стиль бренда.
 Сделай ответ креативным, но без слишком длинных описаний.

17.3. Reasoning task

Мне нужно выбрать модель для backend API, который будет ежедневно обрабатывать 10000 коротких пользовательских запросов.

Сравни критерии выбора модели:

- качество reasoning;
- цена токенов;
- скорость ответа;
- стабильность JSON;
- лимит контекста.

Дай краткую рекомендацию и список рисков.

17.4. Meta-prompt для Cursor/Claude Code

Ты мой AI code-assistant. Нужно сделать проект ai-advent-week1-llm-playground на Python.

Цель проекта: сравнивать ответы LLM при разных параметрах generation.

Требования:

1. Python 3.11+.
2. Конфигурация через .env.
3. CLI-аргументы: --prompt-file, --temperature, --top-p, --max-tokens, --model.
4. Результат сохранять в results/raw_results.json и results/experiments.md.
5. Нельзя коммитить реальные API keys.
6. Сделай README с командами запуска.
7. Добавь обработку ошибок API и пустого prompt.

Сначала создай план файлов, затем реализуй проект полностью.

18. Финальная памятка

- LLM выбирает следующий токен вероятностно; параметры меняют распределение выбора.
- temperature отвечает за степень разнообразия/случайности.
- top_p ограничивает выборку по суммарной вероятности верхних токенов.
- top_k ограничивает выборку количеством самых вероятных токенов.
- max_tokens ограничивает длину вывода, а context length - общий объём контекста.
- seed полезен для экспериментов, но зависит от поддержки провайдера.
- frequency penalty борется с частыми повторами.
- presence penalty стимулирует появление новых тем/токенов.
- system prompt задаёт рамку поведения модели.
- user prompt задаёт конкретную задачу.
- assistant message - ответ модели в истории.
- agenting отличается от обычного prompting наличием цикла perception -> memory -> reasoning -> planning -> action -> feedback.
- Для первой недели лучший результат - простой, но наглядный LLM playground с сохранёнными экспериментами.

Какой вывод сказать в видео

“Я проверил, как одинаковые промпты ведут себя при разных параметрах. Для точных задач лучше низкая temperature и ограниченный вывод. Для creative/brainstorming можно повышать temperature/top_p. max_tokens контролирует длину ответа и стоимость. Для production API важно выбирать модель по балансу качества, скорости и цены.”

19. Источники и основа конспекта

Документ подготовлен на основе загруженного детального конспекта лекции, транскрипции первого занятия недели и вводного занятия AI Advent Challenge #8. Текст переработан: исправлены

смысловые неточности, добавлены практические выводы, чек-листы и шаблоны для выполнения задания.